

Enseignant
Benoit Tremblais
Vincent Verdon

**PORTFOLIO INDIVIDUEL - SAE 302 -
Application graphique de gestion des
sauveteurs spéléologues**

Pape Birame Fall

Rôle : Responsable Backend & Base de Données



1. Portefeuille de Compétences Acquises

Liste des compétences techniques et transversales validées spécifiquement par mes tâches sur le projet.

A. Développement & Base de Données (Cœur du projet)

Compétence	Contexte de mise en œuvre (Ce que j'ai fait)
Conception de Base de Données Relationnelle	Création du schéma SQL (script_base_de_donnee.sql) avec 3 tables liées (Utilisateur, Sauveteur, Planning). Choix des types de données et des contraintes.
Garantie de l'Intégrité des Données	Implémentation de contraintes fortes : Clés Étrangères (FOREIGN KEY) et suppression en cascade (ON DELETE CASCADE) pour éviter les incohérences dans le planning.
Développement d'une couche d'accès aux données	Création de classes Python (PlanningCRUD, SauveteurCRUD) pour isoler les requêtes SQL du reste du code. Protection contre les injections SQL via l'utilisation de paramètres ?.
Maintenance et Évolution du code	Ajout d'une fonctionnalité critique en fin de projet (colonne Lieu dans le planning) impliquant la modification coordonnée du SQL, du Backend et du Contrôleur sans casser l'existant.
Tests Unitaires	Rédaction de tests (tests/test_planning.py) pour valider la persistance des données avant l'intégration graphique.

B. Compétences Transversales

Compétence	Contexte de mise en œuvre
Adaptabilité	Capacité à modifier la structure de la base de données (ajout de la localité) pour répondre à un besoin utilisateur tardif.
Collaboration technique	Fourniture d'une API Python claire (fonctions ajouter_mission, etc.) pour que l'équipe IHM (Interface) puisse travailler sans connaître le SQL.

2. Auto-évaluation sur les Apprentissages Critiques (AC)

N°1. Développer des applications informatiques

Analyse : J'ai structuré le backend en séparant clairement la logique de connexion (`database.py`) et la logique métier (`crud/`). Cette architecture modulaire a permis de développer le backend indépendamment de l'interface graphique.

- **Exemple :** Le fichier `gestion_sauveteurs.py` agit comme un contrôleur central qui reçoit les demandes de l'interface, interroge la BDD via mes classes CRUD, et notifie le réseau. J'ai réussi à faire persister les données localement tout en gérant la réception de données distantes (JSON).

N°2. Administrer des systèmes de données

Analyse : Le choix de SQLite s'est imposé pour la contrainte "hors ligne", mais j'ai dû le configurer rigoureusement (`PRAGMA foreign_keys = ON`) pour qu'il se comporte comme un SGBD robuste. J'ai appris à gérer les transactions pour assurer qu'aucune donnée partielle n'est enregistrée en cas d'erreur.

- **Point fort :** La gestion de la suppression en cascade. Si un gestionnaire supprime un sauveteur, le système nettoie automatiquement le planning. C'est une sécurité que j'ai implémentée directement dans le moteur SQL, rendant l'application plus fiable.

N°3. Travailler en équipe informatique

- **Analyse :** Mon rôle était central car je fournissais les données à l'interface (Sokhna) et je stockais les données reçues du réseau (Perceval). Cela a nécessité une communication constante pour définir le format des dictionnaires Python échangés.
- **Difficulté surmontée :** Lors de l'ajout de la colonne "Lieu", j'ai dû coordonner la mise à jour avec l'équipe pour m'assurer que l'interface envoyait bien le nouveau paramètre et que la base l'acceptait.